

DAOP: Data-Aware Offloading and Predictive Pre-Calculation for Efficient MoE Inference

Yujie Zhang, Shivam Aggarwal, Tulika Mitra
School of Computing, National University of Singapore
{zyujie, shivam, tulika}@comp.nus.edu.sg

Abstract—Mixture-of-Experts (MoE) models, though highly effective for various machine learning tasks, face significant deployment challenges on memory-constrained devices. While GPUs offer fast inference, their limited memory compared to CPUs means not all experts can be stored on the GPU simultaneously, necessitating frequent, costly data transfers from CPU memory, often negating GPU speed advantages. To address this, we present DAOP, an on-device MoE inference engine to optimize parallel GPU-CPU execution. DAOP dynamically allocates experts between CPU and GPU based on per-sequence activation patterns, and selectively pre-calculates predicted experts on CPUs to minimize transfer latency. This approach enables efficient resource utilization across various expert cache ratios while maintaining model accuracy through a novel graceful degradation mechanism. Comprehensive evaluations across various datasets show that DAOP outperforms traditional expert caching and prefetching methods by up to 8.20 $\times$ and offloading techniques by 1.35 $\times$ while maintaining accuracy.

Index Terms—MoE inference engine, data-aware, expert offloading, GPU-CPU hybrid execution

I. INTRODUCTION

Mixture-of-Experts (MoE) architecture [1] addresses the substantial computational demands of Large Language Models (LLMs) by utilizing multiple expert networks and activating only a subset for each input. For instance, in the widely-used Mixtral 8x7B MoE model [2] with 46.6 billion parameters, only 27.4% of the parameters are activated per input sequence (see Fig. 1). This sparse activation balances model capacity with computational efficiency, significantly reducing computational load compared to traditional dense models.

However, deploying these models in low-resource environments is challenging due to limited GPU memory capacity (Fig. 2), which makes it difficult to store all experts in GPU memory and leverage fast execution. Consequently, experts must frequently be loaded from CPU memory, introducing substantial overhead that negates the speed advantages of GPU execution. This data transfer severely undermines inference speed, a critical factor for real-time responsiveness.

Quantization and sparsity techniques [3]–[5] can reduce computational and memory demands but often result in inconsistent performance across various machine learning tasks. Recent strategies like caching frequently activated experts on GPUs and prefetching aim to mitigate lengthy data transfers between CPUs and GPUs [6]–[8]. However, significant delays in computation and expert loading make it challenging to

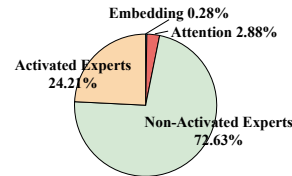


Fig. 1. Average parameter distribution in Mixtral 8x7B [2].

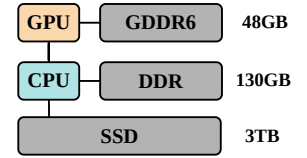


Fig. 2. NVIDIA RTX A6000 GPU specifications.

TABLE I
EXECUTION TIMES (MS)^a FOR TRANSFORMER BLOCK OPERATIONS^b AND EXPERT MIGRATION IN MIXTRAL 8x7B, TESTED ON AN NVIDIA A100 GPU AND AN INTEL XEON GOLD 6326 CPU @ 2.9GHZ, CONNECTED VIA A PCIe 4.0 INTERFACE WITH 64 GB/S BANDWIDTH

Block Execution		Expert Migration (CPU → GPU)	Expert Input / Output Transition (CPU ↔ GPU)
CPU	GPU		
8.02	1.24	39.87	0.02

^aThe time is measured during the decode stage with input/output length 256.

^bEach block includes a self-attention mechanism and expert (linear) layers.

mask the transfer bottlenecks, especially in large MoE models. Innovative strategies are required to utilize CPU and GPU resources to accelerate inference effectively.

As detailed in Table I, on the NVIDIA A100 platform, migrating a single expert within a transformer block from the CPU to the GPU is approximately 32 $\times$ slower than executing the entire block on the GPU with all parameters loaded. This disparity fundamentally limits the efficacy of expert prefetching in concealing memory transfer overhead within the compute-I/O pipeline: prefetching is only effective when host-device transfer time is subsumed by the available computation window, yet large expert weight transfers routinely exceed this window under memory-constrained MoE inference. Conversely, the activations required to be transferred between CPU and GPU for an expert execution on CPU are approximately one ten-thousandth the size of expert weights. Therefore, for an expert that is not present in GPU memory, offloading expert execution to CPU resources is more feasible than loading experts to the GPU for computation.

We introduce *DAOP*, a data-aware MoE inference engine optimized for low-resource devices. DAOP dynamically offloads non-critical experts to CPUs based on inference task requirements, predicting expert activations one layer in advance and selectively pre-calculating them on CPUs using approximate methods. This approach enables DAOP to leverage parallel computing across CPUs and GPUs, significantly accelerating inference with minimal impact on accuracy. Notably, it elimi-

notes the need for model modification or fine-tuning, providing a distinct advantage over task-specific quantization techniques.

Our key contributions are summarised as follows:

- We present and analyze various design choices crucial for large-scale MoE model inference on memory-constrained GPU devices.
- We develop a data-aware MoE inference engine, DAOP, to adaptively and efficiently utilize CPU and GPU resources, thus alleviating GPU memory constraints and I/O bottlenecks.
- We thoroughly evaluate DAOP using popular Mixtral and Phi MoE models across various datasets on NVIDIA RTX A6000. Despite varying numbers of experts cached on the GPU, our algorithm sustains acceptable accuracy and consistently surpasses expert caching and prefetching approaches by $8.20\times$ and offloading methods by $1.35\times$.

II. PRELIMINARIES

A. LLMs with Mixture-of-Experts (MoE)

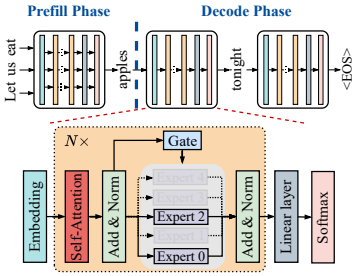


Fig. 3. The decoder-only MoE-based LLM inference procedure with top-2 experts activated per token.

In the prefill stage, tokens embedded with positional information traverse the model to generate the initial output token. The subsequent decode phase employs an autoregressive mechanism to generate each following token sequentially, extending processing time proportionally to the output sequence length. The model comprises multiple transformer blocks, each featuring self-attention, expert layers (feed-forward networks), normalization, and residual connections. Self-attention elucidates inter-word relationships, and expert layers perform non-linear transformations to uncover complex data patterns.

MoE Structure. As illustrated in Fig. 3 (bottom), MoEs activate only a fixed select group of experts (e.g., top-1 or top-2) for each token. The MoE mechanism encompasses *expert selection* and *execution*. A *gating function*, implemented as a multi-layer perceptron (MLP), assesses the hidden states following self-attention to determine each expert’s activation probabilities, indicating their importance to the specific token. Once selected, these experts process and transform the input tokens into output tokens. For clarity, this paper refers to computations within the block, excluding expert selection and computation, as the non-MoE part.

B. Related Work

Edge inference techniques for MoE models often employ expert caching and prefetching strategies. For instance, Mixtral-Offloading [11] dynamically caches expert resources by offloading least used experts to host memory and uploading activated experts to GPU devices, enhanced by mixed quantization to accelerate inference. MoE-Infinity [12] employs activation-aware prefetching to mitigate I/O delays based on sequence-level expert activation patterns. EdgeMoE [13] reduces expert size through expert-specific bit-width adaptation with minimal accuracy loss and preloads anticipated experts using a compute-I/O pipeline. SiDA-MoE [6] uses a hash function to predict expert activation patterns, allowing efficient preloading during runtime. Pre-gated MoE [7] introduces a predictive pre-gating mechanism for optimized prefetching. Despite these advancements, caching and prefetching strategies often struggle to mask expert loading overhead effectively in MoE models with large-scale experts, where limited memory bandwidth remains a bottleneck. Other works, such as Fiddler [14], minimize data movement by executing experts directly on the CPU. Nevertheless, its approach is simplistic and lacks adaptive exploitation of expert activation patterns to enhance model execution efficiency further.

III. OBSERVATIONS & INSIGHTS

This section presents various observations and design choices w.r.t MoE model inference on memory-constrained GPU-CPU devices.

① Dominant experts inherently vary with input sequences.

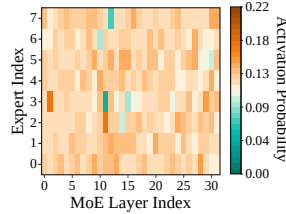


Fig. 4. Layer-wise expert activation pattern of Mixtral 8x7B on dataset C4.

In MoE structures, experts are trained to achieve balanced activation frequencies across various tasks [15]. This balance leads to inherent variability in dominant experts for different inputs, resulting in cache misses and frequent migrations of experts between memory hierarchies. For example, in a comprehensive dataset such as C4, experts are activated with nearly uniform probability across the dataset [4], as illustrated in Fig. 4. This leads to balanced activation within individual input sequences. Alternatively, across different sequences, significant deviations in expert preference frequently occur. Consequently, limited GPU memory struggles to capture activation locality to reduce frequent expert migration. Therefore, adaptive and dynamic offloading mechanisms are essential for efficient execution.

② Expert activation patterns exhibit substantial similarity between prefill and decode phases across input sequences.

To quantify this observation, we use an $L \times E$ matrix to represent these patterns, where L represents the number of MoE layers and E denotes the number of experts per layer. Within this matrix, $P_{i,j}$ and $D_{i,j}$ signify the activation probabilities of expert j at layer i during the prefill and decode stages, respectively. These probabilities are determined by the ratio

TABLE II
AVERAGE SIMILARITY OF EXPERT ACTIVATION MATRICES BETWEEN
PREFILL AND DECODE PHASES IN THE MIXTRAL 8x7B MODEL

	C4	MATH	GSM8K	Average
Similarity (%)	90.05	90.37	91.74	90.72

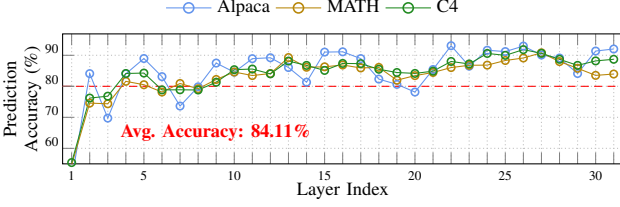


Fig. 5. Layer-wise expert prediction accuracy for the Mixtral 8x7B model, one layer ahead, during the decode phase.

of tokens routed to expert j at layer i to the total tokens processed by that layer. The similarity between the expert activation matrices for two phases is quantified by averaging the cosine similarities of corresponding rows in the two matrices, as detailed as follows:

$$\text{Similarity}(P, D) = \frac{1}{L} \sum_{l=1}^L \text{CosineSimilarity}(P_l, D_l) \quad (1)$$

Table II illustrates the average similarity for the Mixtral 8x7B model, calculated individually for three datasets: C4, MATH, and GSM8K. We randomly select 512 samples for each dataset, with the overall similarity across all datasets averaging approximately 90.72%. Consequently, in our design, the expert activation pattern during the prefill stage informs the optimal distribution of experts between CPU and GPU resources, allowing less-utilized experts to be offloaded to the CPU for approximate computing, thereby minimizing impacts on model accuracy.

③ Expert prediction accuracy is generally high one layer in advance. To forecast the experts needed for the next layer, we apply the subsequent layer’s gating function to the same hidden states employed by the current MoE layer’s gating function. Transformer layers, designed to be residual, naturally enhance each successive layer’s hidden states, providing a solid foundation for predicting future states. Our empirical results, depicted in Fig. 5, reveal that the layer-wise expert prediction accuracy for the Mixtral 8x7B model, when averaged across the Alpaca, MATH, and C4 datasets, stands at 84.11%. These datasets, which span mathematics and world knowledge, underscore the broad applicability of this prediction mechanism. Thus, high prediction accuracy ensures the effectiveness of expert pre-calculation in reducing GPU waiting times.

IV. DAOP: MOE INFERENCE ENGINE

We now outline *DAOP* based on the insights from the observations earlier. Fig. 6 illustrates the critical components: (a) sequence-specific expert allocation between CPUs and GPUs (based on observations ① & ②) and (b) a selective expert pre-calculation strategy driven by predictions without necessitating model fine-tuning (based on observation ③). We

dynamically offload non-dominant experts to CPU memory for each sequence and effectively conceal expert migration overhead during the prefill phase. In the decode phase, we utilize expert parallelism based on prediction to leverage CPU resources, accelerating inference speed.

A. Memory Initialization

We initially allocate non-MoE and dominant experts to the GPU to maximize memory utilization, with others assigned to the CPU. Utilizing the calibration dataset, we identify the expert activation pattern through the decode phase and prioritize experts based on layer-wise activation probabilities for GPU assignment. We standardize the expert cache size across all layers, ensuring it contains experts with the highest activation probabilities. Subsequently, any remaining space—smaller than the layer count—accommodates additional experts, prioritized by their activation frequency.

B. Sequence-specific Expert Allocation

We determine the expert activation pattern for each input sequence during the prefill stage and dynamically adjust expert allocation to optimize reuse during the decode stage. To minimize I/O overhead, expert migration is restricted to the prefill stage for cache adjustments, with this configuration maintained throughout decoding.

Algorithm 1 details our sequence-specific expert allocation method. After the gating function to determine the expert activation pattern in each block (*Line 4*), we count the tokens assigned to each expert, treating this as the expert’s activity level (*Line 7-8*). We generate tuples, grouping the most active CPU experts with the least active GPU experts (*Line 5-9*). If a GPU expert has fewer assigned tokens than its CPU counterpart, we swap their locations (*Line 11-13*). We define a comparison threshold, *SwapInOut*, as 1.05 to avoid unnecessary swaps when the token counts are similar. This process is repeated for all tuples. Finally, we offload less-utilized experts to the CPU and move the most-utilized ones to the GPU. Next, we input multiple tokens from the input sequence to the experts, whether on the CPU or GPU, for parallel local execution (*Line 14*). While assigning multiple tokens to each expert increases GPU utilization and processing speed, it also proportionally raises CPU execution time. Therefore, placing highly utilized experts on GPUs improves inference efficiency and helps mask expert allocation overhead.

C. Prediction-based Expert Pre-Calculation

Fig. 7 illustrates an example computation flow of expert pre-calculation based on prediction during the decode phase. In this stage, the selected experts are executed in parallel in their current locations to minimize time-consuming I/O overhead and accelerate inference, eliminating the need for reallocation. Given the execution time difference between CPUs and GPUs, we predict the experts required for the next layer. When the predicted experts are on the CPU, we selectively pre-calculate their execution using the hidden states from the non-MoE computation in the current layer. If the predicted experts are on the GPU, we feed the hidden states directly from the same

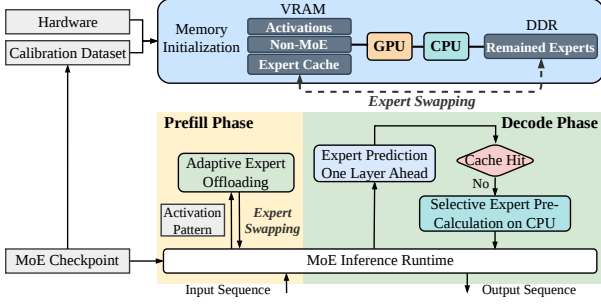


Fig. 6. Design Overview of DAOP.

Algorithm 1: Sequence-specific Expert Allocation

Data: initialized expert cache, model blocks, comparison threshold SwapInOut
Result: updated expert cache, firstly generated output token

```

1 inputTokens = Embedding(inputTokens);
2 foreach block in blocks do
3   inputTokens = executeNonMoE(inputTokens);
4   {RouteInfo} = gating(inputTokens);
5   SwapNum = 0.5 * getNum(block.Experts);
6   {ExpsGPU, ExpsCPU} = findExpertLocs(layer.Experts);
7   HotExps = getTopKActiveExperts(ExpsCPU, SwapNum);
8   ColdExps = getBottomKActiveExperts(ExpsGPU, SwapNum);
9   foreach HotExp, ColdExp in zip(HotExps, ColdExps) do
10    {HotProb, ColdProb} = getActiveProb(HotExp, ColdExp);
11    if HotProb  $\geq$  SwapInOut * ColdProb then
12      swap an expert ColdExp out the expert cache;
13      swap an expert HotExp in the expert cache;
14    local expert execution for tokens in inputTokens in parallel;
15 outputToken = Normalization(inputTokens)

```

layer’s non-MoE execution to the experts. This approach allows CPU-based experts to begin execution immediately after the non-MoE calculation in the previous block, providing sufficient time to parallelize expert execution across both CPU and GPU. Key features of this strategy include:

a) Accuracy Stabilization: As shown in Fig. 5, expert prediction accuracy stabilizes and becomes satisfactory after the first few layers. Consequently, we use the gate function of the $(i + 1)$ -th block on the activations from the non-MoE computation of the i -th block to predict expert selection for block $(i + 1)$, applicable when $i \geq 4$. For blocks where $0 \leq i < 4$, we use the original gate function to determine the activated experts. When the activated experts are already offloaded to CPUs, we transmit the hidden states generated by the non-MoE part to CPUs and send expert computation results back to GPUs, as the experts’ input and output sizes are significantly smaller than the expert parameters.

b) Graceful Degradation: If two predicted experts are on CPUs, we replace the one with the lower score with the next-best expert already on the GPU to minimize overhead. Even if the next-best expert exhibits a lower score, its input data is generated from the non-MoE computation of the current

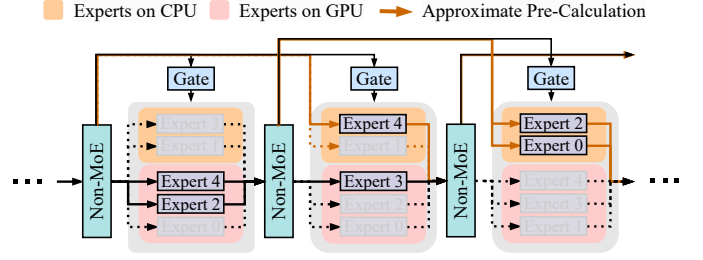


Fig. 7. Computation flow of expert pre-calculation based on prediction.

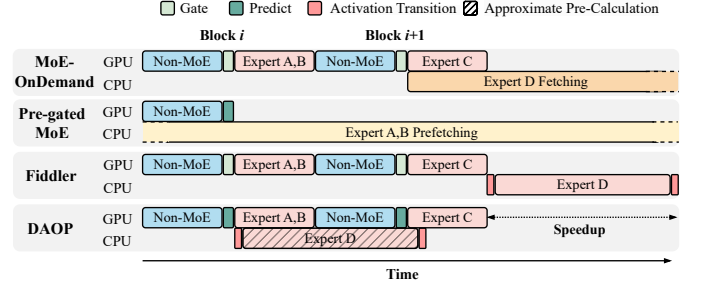


Fig. 8. Execution timeline of DAOP and related works, showing two consecutive Transformer blocks with experts A and B activated for the first block and experts C and D for the second. Initially, experts A, B, and C are cached on GPUs for MoE-OnDemand, Fiddler, and DAOP due to high activation probabilities calibrated.

block, leading to a high contribution to outputs. If no suitable alternative is available, the original selection is maintained for execution, as illustrated in the third block of Fig. 7.

c) GPU-CPU Hybrid Execution: Fig. 8 shows the execution timeline of DAOP and related methods during the decode stage. MoE-OnDemand places non-MoE and dominant experts on the GPU, with the rest stored in host memory. During runtime, experts not on the GPU are migrated dynamically, incurring significant overhead. In both MoE-OnDemand and Pre-gated MoE, fetching or prefetching experts causes severe I/O bottlenecks, as large expert sizes make it difficult to mask I/O latency during block execution. Conversely, Fiddler and DAOP offload expert execution to the CPU when needed experts are absent from the GPU, avoiding migration overhead. DAOP further improves upon Fiddler by leveraging parallel execution between the CPU and GPU, pre-calculating experts on the CPU one layer in advance, thereby accelerating inference. This approach relies on accurate expert prediction, ensuring minimal impact on overall model performance.

V. EXPERIMENTAL EVALUATION

A. Experimental Setup

a) Models and Datasets: We assess the performance of DAOP using two popular decoder-only sparse MoE models, both employing the representative top-2 activation mechanism, as detailed in Table III. This mechanism is extensively employed and considered representative in recently proposed MoE models, as highlighted in a comprehensive survey [16]. The evaluation datasets span several domains, including common-sense reasoning datasets like HellaS (10-shot) [17], Arc-e,

TABLE III
STRUCTURAL DETAILS OF LARGE-SCALE MOE-BASED MODELS

Model	Blocks	Experts	Top-k	Experts Params.	Total Params.
Mixtral 8x7B	32	8	2	45.1B	46.6B
Phi-3.5 MoE	32	16	2	40.3B	41.7B

Arc-c (25-shot) [18], PIQA [19], WinoG (5-shot) [20], and TruthfulQA [21], all in a 0-shot format except where noted. Additionally, we assess popular aggregated results datasets such as MMLU (5-shot) [22] and BBH (3-shot) [23], alongside world knowledge and math datasets including TriviaQA (0-shot) [24] and GSM8K (5-shot) [25]. The evaluation metrics for summarization are Rouge-1 and Rouge-2 scores [26], and for question answering, ExactMatch scores.

b) Hardware: We evaluate the performance of DAOP on an edge platform configured with one Intel i9-10980XE CPU and an NVIDIA RTX A6000 GPU. The CPU includes 18 physical cores at 3.0GHz and 130GB of host memory, while the GPU comprises 48GB GDDR6 with a memory bandwidth of 768GB/s, utilizing a PCIe 4.0 interface with 64GB/s data transfer capability.

c) Implementation: We build our inference engine, DAOP, using the Hugging Face Transformers library [27], a state-of-the-art platform for NLP supporting PyTorch, TensorFlow, and JAX. Our experiments simulate real-time inference scenarios by setting the batch size to one. We integrate DAOP into the evaluation framework [28] to assess model accuracy on downstream tasks discussed previously. In this study, **Expert Cache Ratio (ECR)** is defined as the proportion of available expert slots on the GPU relative to the total number of experts in the model. We adjust this ratio to assess inference speed and accuracy variations, demonstrating DAOP’s scalability across different cache sizes. Using the ShareGPT dataset [29], we calibrate dominant experts for the initial expert cache setup and quantify end-to-end inference performance by the number of tokens generated per second. This dataset is solely for initializing experts and differs from downstream tasks used for accuracy evaluation. For power efficiency, we measure the average power consumption of the entire platform, encompassing both CPU and GPU, using a power and energy monitoring device.

d) Baselines: We evaluate DAOP’s performance against several baselines: MoE-OnDemand, DeepSpeed-MII [30], Mixtral-Offloading [11], and Fiddler [14]. DeepSpeed-MII is a memory-efficient Python library optimized for LLM inference. This library incorporates advanced features such as blocked KV caching and high-performance CUDA kernels to accelerate text generation. Mixtral-Offloading and Fiddler discussed in Section II-B, represent additional comparative frameworks. Pregated MoE [7] is only effective for MoE models with smaller-scale experts and requires substantial fine-tuning; hence, we use MoE-OnDemand as a more practical baseline for direct comparison.

B. Speedup

Fig. 9 compares the inference performance of DAOP against baselines under various input/output sequence length configurations.

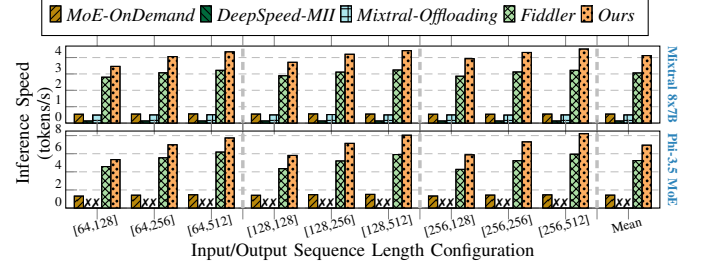


Fig. 9. Inference speed comparison with full GPU memory utilization.

rations. All methods aim to maximize GPU memory utilization, with an ECR of 46.9% for MoE-OnDemand, Fiddler, and DAOP. Due to significant migration overhead, baseline algorithms such as MoE-OnDemand, DeepSpeed-MII, and Mixtral-Offloading achieve less than one token per second for Mixtral 8x7B. Conversely, Fiddler and DAOP, which incur reduced expert migration overhead, demonstrate significant performance improvements that improve with increasing output token length. Notably, DAOP outperforms Fiddler by 40.4%, achieving a token generation rate of 4.52 per second for Mixtral 8x7B and 8.21 per second for Phi-3.5 MoE with an input/output configuration of [256,512].

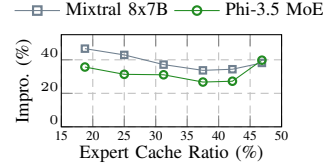


Fig. 10. Inference speed improvement of DAOP over Fiddler for input/output length 256.

Fig. 10 compares the performance of DAOP and Fiddler across various expert cache sizes using an input/output token length of 256. The results show that DAOP consistently achieves an average performance improvement of 35.4% over Fiddler. Even with only 25% of experts cached on the GPU, DAOP achieves a speed of 3.23 tokens/s for the Mixtral 8x7B model and 5.03 tokens/s for the Phi-3.5 MoE model (see Fig. 9 for results at 46.9% ECR).

C. Energy Efficiency

Table IV compares the energy efficiency of DAOP with other baselines, showing that DAOP is the most energy-efficient method evaluated. MoE-OnDemand, DeepSpeed-MII, and Mixtral-Offloading rely solely on the faster GPU for acceleration. However, frequent data transfers from CPU memory cause significant energy consumption, which is particularly severe for DeepSpeed-MII due to its lack of an efficient expert offloading mechanism. Conversely, Fiddler and DAOP leverage CPU and GPU resources for expert execution, reducing excessive expert loading and significantly outperforming these methods. Additionally, DAOP employs selective expert pre-calculation, favoring GPU execution for next-best experts, increasing GPU utilization, and achieving an average 1.50× improvement over Fiddler.

TABLE IV
ENERGY EFFICIENCY (TOKENS/KJ) COMPARISON^a

Model	MoE-OnDemand	DeepSpeed-MII	Mixtral-Offloading	Fiddler	Ours
Mixtral 8x7B	2.63	0.59	2.13	10.06	14.37
Phi-3.5 MoE	6.94	—	—	17.15	27.07

^aFor input/output length 256 with full GPU memory utilization.

TABLE V
IMPACT OF DAOP ON MODEL ACCURACY (%) ACROSS DOWNSTREAM TASKS DEPENDENT ON THE PREFILL STAGE

Model	Method	HellaS	Arc-e	Arc-c	PIQA	WinoG	MMLU
Mixtral 8x7B	Official	66.96	83.10	63.74	83.60	81.69	70.60
	Ours ^a	66.80	84.39	63.82	82.59	81.77	70.47
Phi-3.5 MoE	Official	69.21	76.77	66.64	78.84	78.37	78.78
	Ours ^a	69.25	76.43	66.38	79.00	78.37	78.69

^aOur algorithm utilizes an Expert Cache Ratio (ECR) of 25.0%.

D. Accuracy Results

Table V presents the model accuracy for various tasks using DAOP, focusing on input sequences during the prefill phase, specifically the first output token generated rather than the entire output sequence [28]. The results confirm that our approximate optimizations during decoding do not affect task performance. Additionally, Table VI explores the impact of DAOP on model accuracy throughout the entire inference phase with varying ECRs. The results show minimal impact on accuracy while maintaining efficiency, even with only 37.5% of experts cached on the GPU. The only exception is GSM8K dataset, which we discuss in the next section.

VI. DISCUSSION

A. Platform and Model Applicability

DAOP leverages parallel GPU-CPU execution by adaptively offloading non-critical experts to the CPU for selective pre-computation. Its enhanced inference speed and energy efficiency are based on the following assumptions: 1) GPU memory is limited for storing model weights. 2) The GPU is faster and more energy-efficient for model execution than the CPU. 3) CPU-GPU transfer latency exceeds the time required for expert execution on the CPU. Most commercial GPU devices [31]–[33] satisfy these assumptions, enabling DAOP to provide faster and more energy-efficient inference optimization.

B. Limitation & Future Work

DAOP significantly accelerates inference with minimal accuracy trade-offs across most benchmarks, including commonsense reasoning, text generation, and world knowledge. However, for the GSM8K mathematics dataset, the diverse expert activations within a single sequence limit the capability of a small expert cache to capture activation patterns. We also measure the expert activation pattern variation during the decoding stage for each input sequence using a 15-token window. The results show that the cosine similarity for GSM8K decreases by 3.43% compared to TriviaQA, validating our hypothesis.

TABLE VI
IMPACT OF DAOP ON MODEL ACCURACY (%) ACROSS DOWNSTREAM TASKS DEPENDENT ON ENTIRE INFERENCE STAGE

Model	Method	ECR	TriviaQA	BBH	TruthfulQA		GSM8K
			ExactMatch	ExactMatch	R1 ^a	R2 ^a	ExactMatch
Mixtral 8x7B	Official	100.0%	71.59	49.36	45.04	38.43	58.91
		62.5%	70.98	47.63	46.02	41.74	51.48
		50.0%	70.60	47.10	45.29	41.86	48.07
		37.5%	70.13	47.14	48.10	42.72	41.77
	Ours	25.0%	69.08	46.61	48.47	44.31	33.51
Phi-3.5 MoE	Official	100.0%	—	56.11	56.30	39.66	86.88
		62.5%	—	56.54	54.59	39.53	82.79
		50.0%	—	56.43	53.12	39.53	81.27
		37.5%	—	56.52	56.92	39.41	81.27
	Ours	25.0%	—	56.26	49.45	35.13	74.07

^aR1 and R2 denote the Rouge-1 and Rouge-2 scores, respectively.

VII. CONCLUSION

Our proposed inference engine, DAOP, provides a viable solution for executing MoE models on resource-constrained devices by dynamically offloading non-critical experts to CPUs and selectively pre-calculating based on predictions. This strategy optimizes resource use, mitigating GPU memory constraints without compromising accuracy. In extensive tests with widely used Mixtral and Phi MoE models across diverse datasets, DAOP consistently outperforms traditional expert caching and prefetching techniques by 8.20 $\times$ and offloading methods by 1.35 $\times$. The code is publicly available at: <https://github.com/ecolab-nus/DAOP>.

ACKNOWLEDGMENT

We thank anonymous reviewers for their invaluable feedback that improved the work. This work is supported by the National Research Foundation, Singapore, under its Competitive Research Program Award NRF-CRP23-2019-0003.

REFERENCES

- [1] R. A. Jacobs *et al.*, “Adaptive mixtures of local experts,” *Neural computation*, vol. 3, no. 1, pp. 79–87, 1991.
- [2] A. Q. Jiang *et al.*, “Mixtral of experts,” *arXiv preprint arXiv:2401.04088*, 2024.
- [3] S. Aggarwal *et al.*, “Shedding the bits: Pushing the boundaries of quantization with minifloats on FPGAs,” in *FPL*. IEEE, 2024, pp. 297–303.
- [4] X. Lu *et al.*, “Not all experts are equal: Efficient expert pruning and skipping for mixture-of-experts large language models,” *arXiv preprint arXiv:2402.14800*, 2024.
- [5] S. Aggarwal *et al.*, “CRISP: Hybrid structured sparsity for class-aware model pruning,” in *DATE*, 2024, pp. 1–6.
- [6] Z. Du *et al.*, “SiDA: Sparsity-inspired data-aware serving for efficient and scalable large mixture-of-experts models,” *Proc. of MLSys*, vol. 6, pp. 224–238, 2024.
- [7] R. Hwang *et al.*, “Pre-gated MoE: An algorithm-system co-design for fast and scalable mixture-of-expert inference,” in *ISCA*. IEEE, 2024, pp. 1018–1031.
- [8] S. Zhong *et al.*, “AdapMoE: Adaptive sensitivity-based expert gating and management for efficient moe inference,” *ICCAD*, 2024.
- [9] K. S. Kalyan, “A survey of GPT-3 family large language models including ChatGPT and GPT-4,” *Natural Language Processing Journal*, p. 100048, 2023.
- [10] H. Touvron *et al.*, “LLaMA: Open and efficient foundation language models,” *arXiv preprint arXiv:2302.13971*, 2023.
- [11] A. Eliseev *et al.*, “Fast inference of mixture-of-experts language models with offloading,” *arXiv preprint arXiv:2312.17238*, 2023.
- [12] L. Xue *et al.*, “MoE-Infinity: Activation-aware expert offloading for efficient moe serving,” *arXiv preprint arXiv:2401.14361*, 2024.

- [13] R. Yi *et al.*, “EdgeMoE: Fast on-device inference of moe-based large language models,” *arXiv preprint arXiv:2308.14352*, 2023.
- [14] K. Kamahori *et al.*, “Fiddler: CPU-GPU orchestration for fast inference of mixture-of-experts models,” *arXiv preprint arXiv:2402.07033*, 2024.
- [15] W. Fedus *et al.*, “A review of sparse expert models in deep learning,” *arXiv preprint arXiv:2209.01667*, 2022.
- [16] W. Cai *et al.*, “A survey on mixture of experts in large language models,” *IEEE Transactions on Knowledge and Data Engineering*, p. 1–20, 2025. [Online]. Available: <http://dx.doi.org/10.1109/TKDE.2025.3554028>
- [17] R. Zellers *et al.*, “Hellaswag: Can a machine really finish your sentence?” *arXiv preprint arXiv:1905.07830*, 2019.
- [18] P. Clark *et al.*, “Think you have solved question answering? try arc, the ai2 reasoning challenge,” *arXiv preprint arXiv:1803.05457*, 2018.
- [19] Y. Bisk *et al.*, “PIQA: Reasoning about physical commonsense in natural language,” in *Proc. of AAAI*, vol. 34, no. 05, 2020, pp. 7432–7439.
- [20] K. Sakaguchi *et al.*, “Winogrande: An adversarial winograd schema challenge at scale,” *Communications of the ACM*, vol. 64, no. 9, pp. 99–106, 2021.
- [21] S. Lin *et al.*, “TruthfulQA: Measuring how models mimic human falsehoods,” in *Proceedings of the 60th annual meeting of the association for computational linguistics (volume 1: long papers)*, 2022, pp. 3214–3252.
- [22] D. Hendrycks *et al.*, “Measuring massive multitask language understanding,” *arXiv preprint arXiv:2009.03300*, 2020.
- [23] M. Suzgun *et al.*, “Challenging big-bench tasks and whether chain-of-thought can solve them,” in *Findings of the Association for Computational Linguistics: ACL 2023*, 2023, pp. 13 003–13 051.
- [24] M. Joshi *et al.*, “TriviaQA: A large scale distantly supervised challenge dataset for reading comprehension,” *arXiv preprint arXiv:1705.03551*, 2017.
- [25] K. Cobbe *et al.*, “Training verifiers to solve math word problems,” *arXiv preprint arXiv:2110.14168*, 2021.
- [26] C.-Y. Lin *et al.*, “Rouge: A package for automatic evaluation of summaries,” in *Text summarization branches out*, 2004, pp. 74–81.
- [27] T. Wolf *et al.*, “Transformers: State-of-the-art natural language processing,” in *Proc. of EMNLP: system demonstrations*, 2020, pp. 38–45.
- [28] L. Gao *et al.*, “A framework for few-shot language model evaluation,” 2023. [Online]. Available: <https://zenodo.org/records/10256836>
- [29] S. Team, “ShareGPT,” 2024. [Online]. Available: <https://sharegpt.com/>
- [30] Microsoft, “DeepSpeed-MII,” 2024. [Online]. Available: <https://github.com/microsoft/DeepSpeed-MII>
- [31] NVIDIA, “GeForce RTX 4090,” 2024. [Online]. Available: <https://www.nvidia.com/en-sg/geforce/graphics-cards/40-series/rtx-4090>
- [32] AMD, “MI300X,” 2024. [Online]. Available: <https://www.amd.com/en/products/accelerators/instinct/mi300/mi300x.html>
- [33] J. Choquette, “NVIDIA Hopper H100 GPU: Scaling performance,” *IEEE Micro*, vol. 43, no. 3, pp. 9–17, 2023.